CNNs, Part 2: Training a Convolutional Neural Network

A simple walkthrough of deriving backpropagation for CNNs and implementing it from scratch in Python.

MAY 29, 2019 | UPDATED JANUARY 13, 2026

In this post, we're going to do a deep-dive on something most introductions to Convolutional Neural Networks (CNNs) lack: **how to train a CNN**, including deriving gradients, implementing backprop *from scratch* (using only numpy), and ultimately building a full training pipeline!

is post assumes a basic knowledge of CNNs. My introduction to CNNs (Part 1 of s series) covers everything you need to know, so I'd highly recommend reading that st. If you're here because you've already read Part 1, welcome back!

rts of this post also assume a basic knowledge of multivariable calculus. You n skip those sections if you want, but I recommend reading them even if you don't derstand everything. We'll incrementally write code as we derive results, and even a surface-level understanding can be helpful.

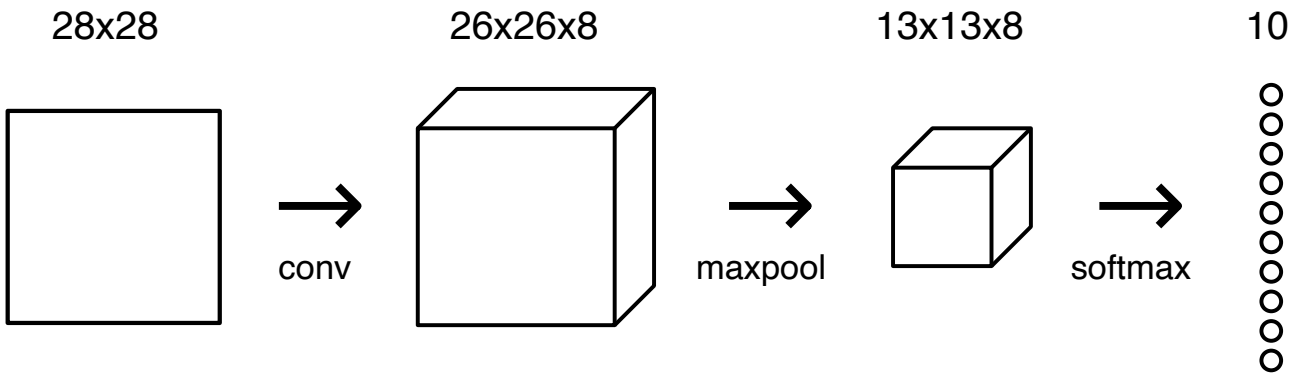
Buckle up! Time to get into it.

1. Setting the Stage

We'll pick back up where Part 1 of this series left off. We were using a CNN to tackle the MNIST handwritten digit classification problem:

Sample images from the MNIST dataset

Our (simple) CNN consisted of a Conv layer, a Max Pooling layer, and a Softmax layer. Here's that diagram of our CNN again:



We'd written 3 classes, one for each layer: `Conv3x3`, `MaxPool`, and `Softmax`. Each class implemented a `forward()` method that we used to build the forward pass of the CNN:

```
# Header: cnn.py
conv = Conv3x3(8)                  # 28x28x1 -> 26x26x8
pool = MaxPool2()                  # 26x26x8 -> 13x13x8
softmax = Softmax(13 * 13 * 8, 10) # 13x13x8 -> 10

def forward(image, label):
```

```
  - image is a 2d numpy array
  - label is a digit
  '''
  # We transform the image from [0, 255] to [-0.5, 0.5] to make it easier
  # to work with. This is standard practice.
  out = conv.forward((image / 255) - 0.5)
  out = pool.forward(out)
  out = softmax.forward(out)

  # Calculate cross-entropy loss and accuracy. np.log() is the natural log.
  loss = -np.log(out[label])
  acc = 1 if np.argmax(out) == label else 0

  return out, loss, acc
```

You can **view the code or run the CNN in your browser**. It's also available on Github.

Here's what the output of our CNN looks like right now:

```
MNIST CNN initialized!
[Step 100] Past 100 steps: Average Loss 2.302 | Accuracy: 11%
[Step 200] Past 100 steps: Average Loss 2.302 | Accuracy: 8%
[Step 300] Past 100 steps: Average Loss 2.302 | Accuracy: 3%
[Step 400] Past 100 steps: Average Loss 2.302 | Accuracy: 12%
```

Obviously, we'd like to do better than 10% accuracy... let's teach this CNN a lesson.

2. Training Overview

Training a neural network typically consists of two phases:

1. A **forward** phase, where the input is passed completely through the network.
2. A **backward** phase, where gradients are backpropagated (backprop) and weights are updated.

We'll follow this pattern to train our CNN. There are also two major implementation-specific ideas we'll use:

phase must be preceded by a corresponding forward phase.

- During the backward phase, each layer will **receive a gradient** and also **return a gradient**. It will receive the gradient of loss with respect to its *outputs* ($\frac{\partial L}{\partial \text{out}}$) and return the gradient of loss with respect to its *inputs* ($\frac{\partial L}{\partial \text{in}}$).

These two ideas will help keep our training implementation clean and organized. The best way to see why is probably by looking at code. Training our CNN will ultimately look something like this:

```
# Feed forward
out = conv.forward((image / 255) - 0.5)
out = pool.forward(out)
out = softmax.forward(out)

# Calculate initial gradient
gradient = np.zeros(10)
# ...

# Backprop
gradient = softmax.backprop(gradient)
gradient = pool.backprop(gradient)
gradient = conv.backprop(gradient)
```

See how nice and clean that looks? Now imagine building a network with 50 layers instead of 3 - it's even more valuable then to have good systems in place.

3. Backprop: Softmax

We'll start our way from the end and work our way towards the beginning, since that's how backprop works. First, recall the **cross-entropy loss**:

$$L = -\ln(p_c)$$

where p_c is the predicted probability for the correct class c (in other words, what digit our current image *actually* is).

The first thing we need to calculate is the input to the Softmax layer's backward phase, $\frac{\partial L}{\partial out_s}$, where out_s is the output from the Softmax layer: a vector of 10 probabilities. This is pretty easy, since only p_i shows up in the loss equation:

$$\frac{\partial L}{\partial out_s(i)} = \begin{cases} 0 & \text{if } i \neq c \\ -\frac{1}{p_i} & \text{if } i = c \end{cases}$$

Reminder: c is the correct class.

That's our initial gradient you saw referenced above:

```
# Calculate initial gradient
gradient = np.zeros(10)
gradient[label] = -1 / out[label]
```

We're almost ready to implement our first backward phase - we just need to first perform the forward phase caching we discussed earlier:

```
# Header: softmax.py
class Softmax:
  # ...

  def forward(self, input):
    '''
    Performs a forward pass of the softmax layer using the given input.
    Returns a 1d numpy array containing the respective probability values.
    - input can be any array with any dimensions.
    '''
    self.last_input_shape = input.shape

    input = input.flatten()
    self.last_input = input

    input_len, nodes = self.weights.shape

    totals = np.dot(input, self.weights) + self.biases
    self.last_totals = totals
```

We cache 3 things here that will be useful for implementing the backward phase:

- The `input` 's shape *before* we flatten it.
- The `input` *after* we flatten it.
- The **totals**, which are the values passed in to the softmax activation.

With that out of the way, we can start deriving the gradients for the backprop phase. We've already derived the input to the Softmax backward phase: $\frac{\partial L}{\partial out_s}$. One fact we can use about $\frac{\partial L}{\partial out_s}$ is that *it's only nonzero for c, the correct class*. That means that we can ignore everything but $out_s(c)$!

First, let's calculate the gradient of $out_s(c)$ with respect to the totals (the values passed in to the softmax activation). Let t_i be the total for class i. Then we can write $out_s(c)$ as:

$$out_s(c) = \frac{e^{t_c}}{\sum_i e^{t_i}} = \frac{e^{t_c}}{S}$$

where $S = \sum_i e^{t_i}$.

> *Need a refresher on Softmax? Read my simple explanation of Softmax.*

Now, consider some class k such that $k \neq c$. We can rewrite $out_s(c)$ as:

$$out_s(c) = e^{t_c} S^{-1}$$

and use Chain Rule to derive:

$$
\begin{aligned}
&= -e^{t_c} S^{-2} (\frac{\partial S}{\partial t_k}) \\
&= -e^{t_c} S^{-2} (e^{t_k}) \\
&= \boxed{\frac{-e^{t_c} e^{t_k}}{S^2}}
\end{aligned}
$$

Remember, that was assuming $k \neq c$. Now let's do the derivation for c, this time using Quotient Rule (because we have an e^{t_c} in the numerator of $out_s(c)$):

$$
\begin{aligned}
\frac{\partial out_s(c)}{\partial t_c} &= \frac{S e^{t_c} - e^{t_c} \frac{\partial S}{\partial t_c}}{S^2} \\
&= \frac{S e^{t_c} - e^{t_c} e^{t_c}}{S^2} \\
&= \boxed{\frac{e^{t_c}(S - e^{t_c})}{S^2}}
\end{aligned}
$$

Phew. That was the hardest bit of calculus in this entire post - it only gets easier from here! Let's start implementing this:

```
# Header: softmax.py
class Softmax:
  # ...

  def backprop(self, d_L_d_out):
    '''
    Performs a backward pass of the softmax layer.
    Returns the loss gradient for this layer's inputs.
    - d_L_d_out is the loss gradient for this layer's outputs.
    '''
    # We know only 1 element of d_L_d_out will be nonzero
    for i, gradient in enumerate(d_L_d_out):
      if gradient == 0:
        continue

      # e^totals
      t_exp = np.exp(self.last_totals)

      # Sum of all e^totals
      S = np.sum(t_exp)
```

```
d_out_d_t[i] = t_exp[i] * (S - t_exp[i]) / (S ** 2)

# ... to be continued
```

Remember how $\frac{\partial L}{\partial out_s}$ is only nonzero for the correct class, c? We start by looking for c by looking for a nonzero gradient in `d_L_d_out`. Once we find that, we calculate the gradient $\frac{\partial out_s(i)}{\partial t}$ (`d_out_d_totals`) using the results we derived above:

$$\frac{\partial out_s(k)}{\partial t} = \begin{cases} \frac{-e^{t_c} e^{t_k}}{S^2} & \text{if } k \neq c \\ \frac{e^{t_c}(S - e^{t_c})}{S^2} & \text{if } k = c \end{cases}$$

Let's keep going. We ultimately want the gradients of loss against weights, biases, and input:

- We'll use the weights gradient, $\frac{\partial L}{\partial w}$, to update our layer's weights.
- We'll use the biases gradient, $\frac{\partial L}{\partial b}$, to update our layer's biases.
- We'll return the input gradient, $\frac{\partial L}{\partial input}$, from our `backprop()` method so the next layer can use it. This is the return gradient we talked about in the Training Overview section!

To calculate those 3 loss gradients, we first need to derive 3 more results: the gradients of *totals* against weights, biases, and input. The relevant equation here is:

$$t = w * input + b$$

These gradients are easy!

$$\frac{\partial t}{\partial w} = input$$

$$\frac{\partial t}{\partial b} = 1$$

$$\frac{\partial t}{\partial input} = w$$

Putting everything together:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial out} * \frac{\partial out}{\partial t} * \frac{\partial t}{\partial b}$$

$$\frac{\partial L}{\partial input} = \frac{\partial L}{\partial out} * \frac{\partial out}{\partial t} * \frac{\partial t}{\partial input}$$

Putting this into code is a little less straightforward:

```
# Header: softmax.py
class Softmax:
  # ...

  def backprop(self, d_L_d_out):
    '''
    Performs a backward pass of the softmax layer.
    Returns the loss gradient for this layer's inputs.
    - d_L_d_out is the loss gradient for this layer's outputs.
    '''
    # We know only 1 element of d_L_d_out will be nonzero
    for i, gradient in enumerate(d_L_d_out):
      if gradient == 0:
        continue

      # e^totals
      t_exp = np.exp(self.last_totals)

      # Sum of all e^totals
      S = np.sum(t_exp)

      # Gradients of out[i] against totals
      d_out_d_t = -t_exp[i] * t_exp / (S ** 2)
      d_out_d_t[i] = t_exp[i] * (S - t_exp[i]) / (S ** 2)

      # Gradients of totals against weights/biases/input
      d_t_d_w = self.last_input
      d_t_d_b = 1
      d_t_d_inputs = self.weights
      # Gradients of loss against totals
      d_L_d_t = gradient * d_out_d_t
      # Gradients of loss against weights/biases/input
      d_L_d_w = d_t_d_w[np.newaxis].T @ d_L_d_t[np.newaxis]
      d_L_d_b = d_L_d_t * d_t_d_b
      d_L_d_inputs = d_t_d_inputs @ d_L_d_t
```

First, we pre-calculate `d_L_d_t` since we'll use it several times. Then, we calculate each gradient:

- **`d_L_d_w`**: We need 2d arrays to do matrix multiplication (`@`), but `d_t_d_w` and `d_L_d_t` are 1d arrays. np.newaxis lets us easily create a new axis of length one, so we end up multiplying matrices with dimensions (`input_len`, 1) and (1, `nodes`). Thus, the final result for `d_L_d_w` will have shape (`input_len`, `nodes`), which is the same as `self.weights`!
- **`d_L_d_b`**: This one is straightforward, since `d_t_d_b` is 1.
- **`d_L_d_inputs`**: We multiply matrices with dimensions (`input_len`, `nodes`) and (`nodes`, 1) to get a result with length `input_len`.

> *Try working through small examples of the calculations above, especially the matrix multiplications for `d_L_d_w` and `d_L_d_inputs`. That's the best way to understand why this code correctly computes the gradients.*

With all the gradients computed, all that's left is to actually train the Softmax layer! We'll update the weights and bias using Stochastic Gradient Descent (SGD) just like we did in my introduction to Neural Networks and then return `d_L_d_inputs`:

```
# Header: softmax.py
class Softmax
  # ...

  def backprop(self, d_L_d_out, learn_rate):
    '''
    Performs a backward pass of the softmax layer.
    Returns the loss gradient for this layer's inputs.
    - d_L_d_out is the loss gradient for this layer's outputs.
    - learn_rate is a float    '''
    # We know only 1 element of d_L_d_out will be nonzero
    for i, gradient in enumerate(d_L_d_out):
      if gradient == 0:
        continue

      # e^totals
```

```
    S = np.sum(t_exp)

    # Gradients of out[i] against totals
    d_out_d_t = -t_exp[i] * t_exp / (S ** 2)
    d_out_d_t[i] = t_exp[i] * (S - t_exp[i]) / (S ** 2)

    # Gradients of totals against weights/biases/input
    d_t_d_w = self.last_input
    d_t_d_b = 1
    d_t_d_inputs = self.weights

    # Gradients of loss against totals
    d_L_d_t = gradient * d_out_d_t

    # Gradients of loss against weights/biases/input
    d_L_d_w = d_t_d_w[np.newaxis].T @ d_L_d_t[np.newaxis]
    d_L_d_b = d_L_d_t * d_t_d_b
    d_L_d_inputs = d_t_d_inputs @ d_L_d_t

    # Update weights / biases
    self.weights -= learn_rate * d_L_d_w
    self.biases -= learn_rate * d_L_d_b
    return d_L_d_inputs.reshape(self.last_input_shape)
```

Notice that we added a `learn_rate` parameter that controls how fast we update our weights. Also, we have to `reshape()` before returning `d_L_d_inputs` because we flattened the input during our forward pass:

```
# Header: softmax.py
class Softmax:
  # ...

  def forward(self, input):
    '''
    Performs a forward pass of the softmax layer using the given input.
    Returns a 1d numpy array containing the respective probability values.
    - input can be any array with any dimensions.
    '''
    self.last_input_shape = input.shape

    input = input.flatten()
    self.last_input = input
```

Reshaping to `last_input_shape` ensures that this layer returns gradients for its input in the same format that the input was originally given to it.

Test Drive: Softmax Backprop

We've finished our first backprop implementation! Let's quickly test it to see if it's any good. We'll start implementing a `train()` method in our `cnn.py` file from Part 1:

```
# Header: cnn.py
# Imports and setup here
# ...

def forward(image, label):
  # Implementation excluded
  # ...

def train(im, label, lr=.005):
  '''
  Completes a full training step on the given image and label.
  Returns the cross-entropy loss and accuracy.
  - image is a 2d numpy array
  - label is a digit
  - lr is the learning rate
  '''
  # Forward
  out, loss, acc = forward(im, label)

  # Calculate initial gradient
  gradient = np.zeros(10)
  gradient[label] = -1 / out[label]

  # Backprop
  gradient = softmax.backprop(gradient, lr)
  # TODO: backprop MaxPool2 layer
  # TODO: backprop Conv3x3 layer

  return loss, acc

print('MNIST CNN initialized!')
```

```
for i, (im, label) in enumerate(zip(train_images, train_labels)):
  if i % 100 == 99:
    print(
      '[Step %d] Past 100 steps: Average Loss %.3f | Accuracy: %d%%' %
      (i + 1, loss / 100, num_correct)
    )
    loss = 0
    num_correct = 0

  l, acc = train(im, label)
  loss += l
  num_correct += acc
```

Running this gives results similar to:

```
MNIST CNN initialized!
[Step 100] Past 100 steps: Average Loss 2.239 | Accuracy: 18%
[Step 200] Past 100 steps: Average Loss 2.140 | Accuracy: 32%
[Step 300] Past 100 steps: Average Loss 1.998 | Accuracy: 48%
[Step 400] Past 100 steps: Average Loss 1.861 | Accuracy: 59%
[Step 500] Past 100 steps: Average Loss 1.789 | Accuracy: 56%
[Step 600] Past 100 steps: Average Loss 1.809 | Accuracy: 48%
[Step 700] Past 100 steps: Average Loss 1.718 | Accuracy: 63%
[Step 800] Past 100 steps: Average Loss 1.588 | Accuracy: 69%
[Step 900] Past 100 steps: Average Loss 1.509 | Accuracy: 71%
[Step 1000] Past 100 steps: Average Loss 1.481 | Accuracy: 70%
```

The loss is going down and the accuracy is going up - our CNN is already learning!

4. Backprop: Max Pooling

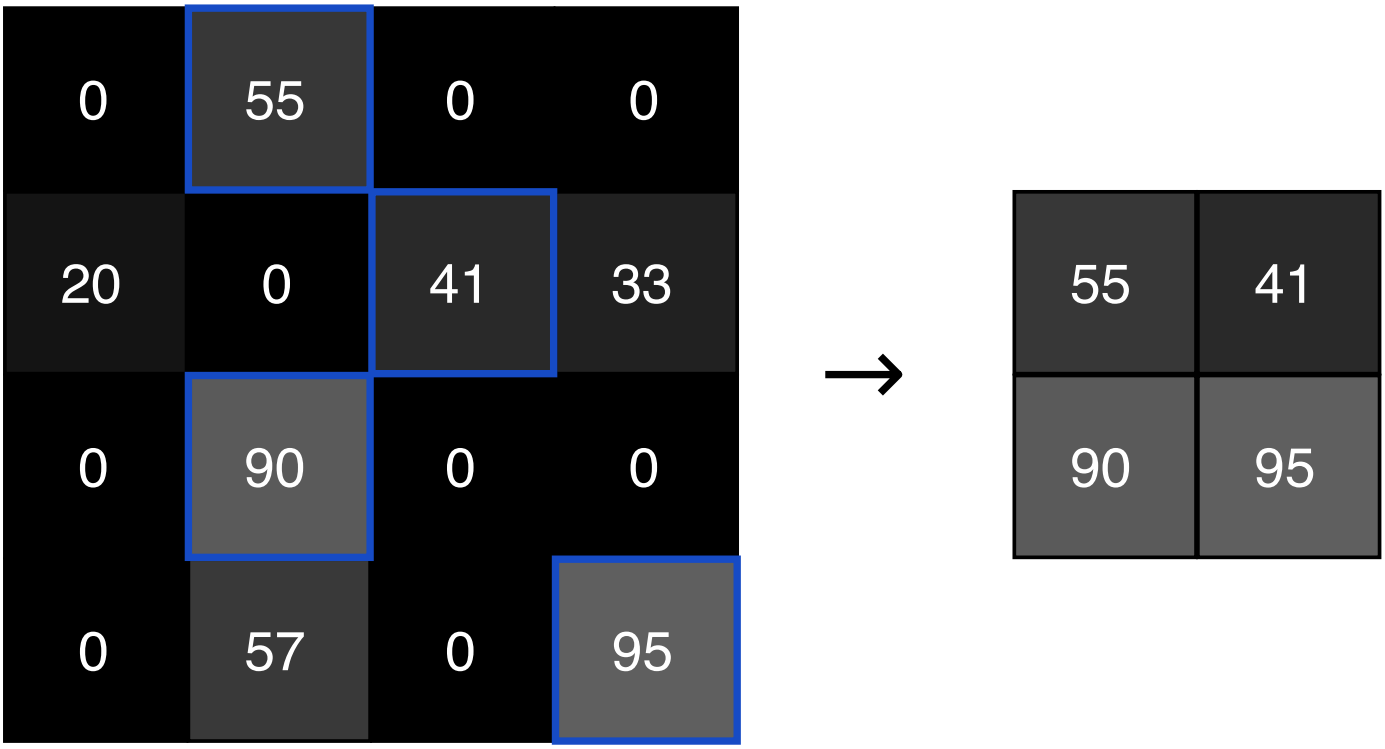
A Max Pooling layer can't be trained because it doesn't actually have any weights, but we still need to implement a `backprop()` method for it to calculate gradients. We'll start by adding forward phase caching again. All we need to cache this time is the input:

```
# Header: maxpool.py
class MaxPool2:
```

```
    '''
    Performs a forward pass of the maxpool layer using the given input.
    Returns a 3d numpy array with dimensions (h / 2, w / 2, num_filters).
    - input is a 3d numpy array with dimensions (h, w, num_filters)
    '''
    self.last_input = input

    # More implementation
    # ...
```

During the forward pass, the Max Pooling layer takes an input volume and halves its width and height dimensions by picking the max values over 2x2 blocks. The backward pass does the opposite: **we'll double the width and height** of the loss gradient by assigning each gradient value to **where the original max value was** in its corresponding 2x2 block.

Here's an example. Consider this forward phase for a Max Pooling layer:



The backward phase of that same layer would look like this:

Each gradient value is assigned to where the original max value was, and every other value is zero.

Why does the backward phase for a Max Pooling layer work like this? Think about what $\frac{\partial L}{\partial inputs}$ intuitively should be. An input pixel that isn't the max value in its 2x2 block would have *zero* marginal effect on the loss, because changing that value slightly wouldn't change the output at all! In other words, $\frac{\partial L}{\partial input} = 0$ for non-max pixels. On the other hand, an input pixel that *is* the max value would have its value passed through to the output, so $\frac{\partial output}{\partial input} = 1$, meaning $\frac{\partial L}{\partial input} = \frac{\partial L}{\partial output}$.

We can implement this pretty quickly using the `iterate_regions()` helper method we [wrote in Part 1](). I'll include it again as a reminder:

```
# Header: maxpool.py
class MaxPool2:
  # ...

  def iterate_regions(self, image):
    '''
    Generates non-overlapping 2x2 image regions to pool over.
    - image is a 2d numpy array
    '''
    h, w, _ = image.shape
    new_h = h // 2
```

```
      for j in range(new_w):
        im_region = image[(i * 2):(i * 2 + 2), (j * 2):(j * 2 + 2)]
        yield im_region, i, j

  def backprop(self, d_L_d_out):
    '''
    Performs a backward pass of the maxpool layer.
    Returns the loss gradient for this layer's inputs.
    - d_L_d_out is the loss gradient for this layer's outputs.
    '''
    d_L_d_input = np.zeros(self.last_input.shape)

    for im_region, i, j in self.iterate_regions(self.last_input):
      h, w, f = im_region.shape
      amax = np.amax(im_region, axis=(0, 1))

      for i2 in range(h):
        for j2 in range(w):
          for f2 in range(f):
            # If this pixel was the max value, copy the gradient to it.
            if im_region[i2, j2, f2] == amax[f2]:
              d_L_d_input[i * 2 + i2, j * 2 + j2, f2] = d_L_d_out[i, j, f2]

    return d_L_d_input
```

For each pixel in each 2x2 image region in each filter, we copy the gradient from `d_L_d_out` to `d_L_d_input` if it was the max value during the forward pass.

That's it! On to our final layer.

5. Backprop: Conv

We're finally here: backpropagating through a Conv layer is the core of training a CNN. The forward phase caching is simple:

```
# Header: conv.py
class Conv3x3
  # ...
```

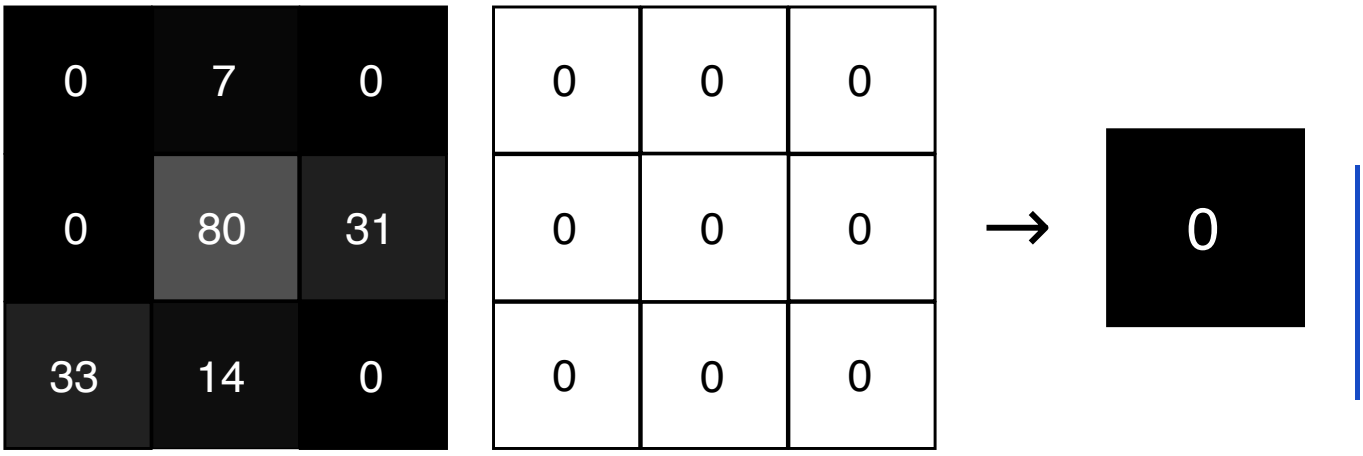
```
    Returns a 3d numpy array with dimensions (h, w, num_filters).
    - input is a 2d numpy array
    '''
    self.last_input = input

    # More implementation
    # ...
```

> *Reminder about our implementation: for simplicity,* ***we assume the input to our conv layer is a 2d array****. This only works for us because we use it as the first layer in our network. If we were building a bigger network that needed to use* `Conv3x3` *multiple times, we'd have to make the input be a* ***3d*** *array.*

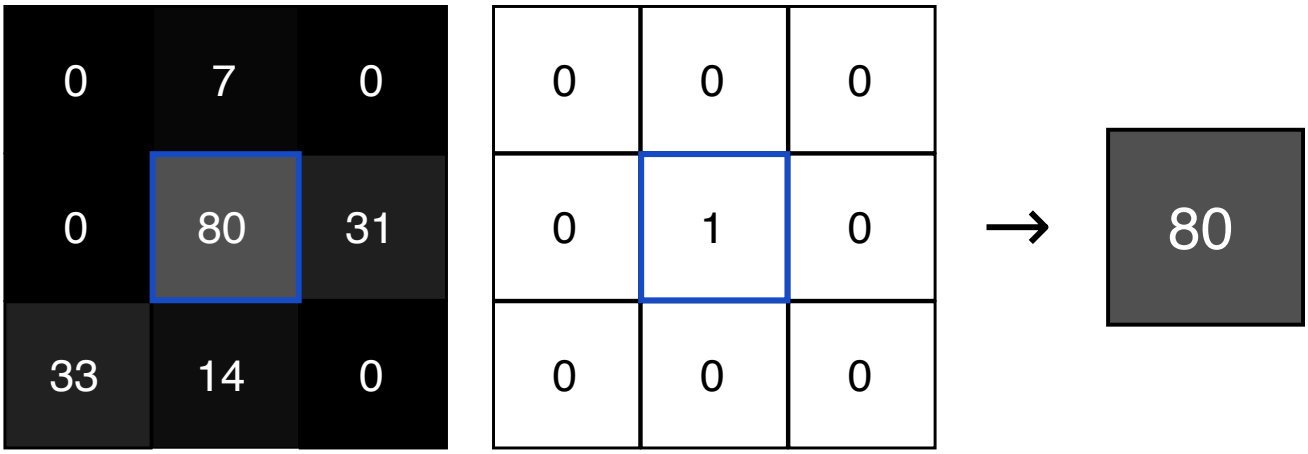
We're primarily interested in the loss gradient for the filters in our conv layer, since we need that to update our filter weights. We already have $\frac{\partial L}{\partial out}$ for the conv layer, so we just need $\frac{\partial out}{\partial filters}$. To calculate that, we ask ourselves this: how would changing a filter's weight affect the conv layer's output?

The reality is that **changing any filter weights would affect the *entire* output image** for that filter, since *every* output pixel uses *every* pixel weight during convolution. To make this even easier to think about, let's just think about one output pixel at a time: **how would modifying a filter change the output of *one* specific output pixel?**

Here's a super simple example to help think about this question:



center image value, 80:



Similarly, increasing any of the other filter weights by 1 would increase the output by the value of the corresponding image pixel! This suggests that the derivative of a specific output pixel with respect to a specific filter weight is just the corresponding image pixel value. Doing the math confirms this:

$$\begin{aligned} \text{out(i, j)} &= \text{convolve(image, filter)} \\ &= \sum_{x=0}^{3}\sum_{y=0}^{3} \text{image}(i+x, j+y) * \text{filter}(x, y) \end{aligned}$$

$$\frac{\partial \text{out}(i,j)}{\partial \text{filter}(x,y)} = \text{image}(i+x, j+y)$$

We can put it all together to find the loss gradient for specific filter weights:

$$\frac{\partial L}{\partial \text{filter}(x,y)} = \sum_i \sum_j \frac{\partial L}{\partial \text{out}(i,j)} * \frac{\partial \text{out}(i,j)}{\partial \text{filter}(x,y)}$$

We're ready to implement backprop for our conv layer!

```
# Header: conv.py
class Conv3x3
  # ...

  def backprop(self, d_L_d_out, learn_rate):
```

```
    - learn_rate is a float.
    '''
    d_L_d_filters = np.zeros(self.filters.shape)

    for im_region, i, j in self.iterate_regions(self.last_input):
      for f in range(self.num_filters):
        d_L_d_filters[f] += d_L_d_out[i, j, f] * im_region

    # Update filters
    self.filters -= learn_rate * d_L_d_filters

    # We aren't returning anything here since we use Conv3x3 as
    # the first layer in our CNN. Otherwise, we'd need to return
    # the loss gradient for this layer's inputs, just like every
    # other layer in our CNN.
    return None
```

We apply our derived equation by iterating over every image region / filter and incrementally building the loss gradients. Once we've covered everything, we update `self.filters` using SGD just as before. Note the comment explaining why we're returning `None` - the derivation for the loss gradient of the inputs is very similar to what we just did and is left as an exercise to the reader :).

With that, we're done! We've implemented a full backward pass through our CNN. Time to test it out...

6. Training a CNN

We'll train our CNN for a few epochs, track its progress during training, and then test it on a separate test set. Here's the full code:

```
# Header: cnn.py
import mnist
import numpy as np
from conv import Conv3x3
from maxpool import MaxPool2
from softmax import Softmax
```

```
train_labels = mnist.train_labels()[:1000]
test_images = mnist.test_images()[:1000]
test_labels = mnist.test_labels()[:1000]

conv = Conv3x3(8)                    # 28x28x1 -> 26x26x8
pool = MaxPool2()                    # 26x26x8 -> 13x13x8
softmax = Softmax(13 * 13 * 8, 10) # 13x13x8 -> 10

def forward(image, label):
  '''
  Completes a forward pass of the CNN and calculates the accuracy and
  cross-entropy loss.
  - image is a 2d numpy array
  - label is a digit
  '''
  # We transform the image from [0, 255] to [-0.5, 0.5] to make it easier
  # to work with. This is standard practice.
  out = conv.forward((image / 255) - 0.5)
  out = pool.forward(out)
  out = softmax.forward(out)

  # Calculate cross-entropy loss and accuracy. np.log() is the natural log.
  loss = -np.log(out[label])
  acc = 1 if np.argmax(out) == label else 0

  return out, loss, acc

def train(im, label, lr=.005):
  '''
  Completes a full training step on the given image and label.
  Returns the cross-entropy loss and accuracy.
  - image is a 2d numpy array
  - label is a digit
  - lr is the learning rate
  '''
  # Forward
  out, loss, acc = forward(im, label)

  # Calculate initial gradient
  gradient = np.zeros(10)
  gradient[label] = -1 / out[label]

  # Backprop
```

```
  return loss, acc

print('MNIST CNN initialized!')

# Train the CNN for 3 epochs
for epoch in range(3):
  print('--- Epoch %d ---' % (epoch + 1))

  # Shuffle the training data
  permutation = np.random.permutation(len(train_images))
  train_images = train_images[permutation]
  train_labels = train_labels[permutation]

  # Train!
  loss = 0
  num_correct = 0
  for i, (im, label) in enumerate(zip(train_images, train_labels)):
    if i > 0 and i % 100 == 99:
      print(
        '[Step %d] Past 100 steps: Average Loss %.3f | Accuracy: %d%%' %
        (i + 1, loss / 100, num_correct)
      )
      loss = 0
      num_correct = 0

    l, acc = train(im, label)
    loss += l
    num_correct += acc

# Test the CNN
print('\n--- Testing the CNN ---')
loss = 0
num_correct = 0
for im, label in zip(test_images, test_labels):
  _, l, acc = forward(im, label)
  loss += l
  num_correct += acc

num_tests = len(test_images)
print('Test Loss:', loss / num_tests)
print('Test Accuracy:', num_correct / num_tests)
```

```
MNIST CNN initialized!
--- Epoch 1 ---
[Step 100] Past 100 steps: Average Loss 2.254 | Accuracy: 18%
[Step 200] Past 100 steps: Average Loss 2.167 | Accuracy: 30%
[Step 300] Past 100 steps: Average Loss 1.676 | Accuracy: 52%
[Step 400] Past 100 steps: Average Loss 1.212 | Accuracy: 63%
[Step 500] Past 100 steps: Average Loss 0.949 | Accuracy: 72%
[Step 600] Past 100 steps: Average Loss 0.848 | Accuracy: 74%
[Step 700] Past 100 steps: Average Loss 0.954 | Accuracy: 68%
[Step 800] Past 100 steps: Average Loss 0.671 | Accuracy: 81%
[Step 900] Past 100 steps: Average Loss 0.923 | Accuracy: 67%
[Step 1000] Past 100 steps: Average Loss 0.571 | Accuracy: 83%
--- Epoch 2 ---
[Step 100] Past 100 steps: Average Loss 0.447 | Accuracy: 89%
[Step 200] Past 100 steps: Average Loss 0.401 | Accuracy: 86%
[Step 300] Past 100 steps: Average Loss 0.608 | Accuracy: 81%
[Step 400] Past 100 steps: Average Loss 0.511 | Accuracy: 83%
[Step 500] Past 100 steps: Average Loss 0.584 | Accuracy: 89%
[Step 600] Past 100 steps: Average Loss 0.782 | Accuracy: 72%
[Step 700] Past 100 steps: Average Loss 0.397 | Accuracy: 84%
[Step 800] Past 100 steps: Average Loss 0.560 | Accuracy: 80%
[Step 900] Past 100 steps: Average Loss 0.356 | Accuracy: 92%
[Step 1000] Past 100 steps: Average Loss 0.576 | Accuracy: 85%
--- Epoch 3 ---
[Step 100] Past 100 steps: Average Loss 0.367 | Accuracy: 89%
[Step 200] Past 100 steps: Average Loss 0.370 | Accuracy: 89%
[Step 300] Past 100 steps: Average Loss 0.464 | Accuracy: 84%
[Step 400] Past 100 steps: Average Loss 0.254 | Accuracy: 95%
[Step 500] Past 100 steps: Average Loss 0.366 | Accuracy: 89%
[Step 600] Past 100 steps: Average Loss 0.493 | Accuracy: 89%
[Step 700] Past 100 steps: Average Loss 0.390 | Accuracy: 91%
[Step 800] Past 100 steps: Average Loss 0.459 | Accuracy: 87%
[Step 900] Past 100 steps: Average Loss 0.316 | Accuracy: 92%
[Step 1000] Past 100 steps: Average Loss 0.460 | Accuracy: 87%

--- Testing the CNN ---
Test Loss: 0.5979384893783474
Test Accuracy: 0.78
```

Our code works! In only 3000 training steps, we went from a model with 2.3 loss and 10% accuracy to 0.6 loss and 78% accuracy.

We only used a subset of the entire MNIST dataset for this example in the interest of time - our CNN implementation isn't particularly fast. If we wanted to train a MNIST CNN for real, we'd use an ML library like Keras. To illustrate the power of our CNN, I used Keras to implement and train the *exact same* CNN we just built from scratch:

```
# Header: cnn_keras.py
import numpy as np
import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import SGD

train_images = mnist.train_images()
train_labels = mnist.train_labels()
test_images = mnist.test_images()
test_labels = mnist.test_labels()

train_images = (train_images / 255) - 0.5
test_images = (test_images / 255) - 0.5

train_images = np.expand_dims(train_images, axis=3)
test_images = np.expand_dims(test_images, axis=3)

model = Sequential([
  Conv2D(8, 3, input_shape=(28, 28, 1), use_bias=False),
  MaxPooling2D(pool_size=2),
  Flatten(),
  Dense(10, activation='softmax'),
])

model.compile(SGD(lr=.005), loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(
  train_images,
  to_categorical(train_labels),
  batch_size=1,
  epochs=3,
  validation_data=(test_images, to_categorical(test_labels)),
)
```

```
Epoch 1
loss: 0.2433 - acc: 0.9276 - val_loss: 0.1176 - val_acc: 0.9634
Epoch 2
loss: 0.1184 - acc: 0.9648 - val_loss: 0.0936 - val_acc: 0.9721
Epoch 3
loss: 0.0930 - acc: 0.9721 - val_loss: 0.0778 - val_acc: 0.9744
```

We achieve **97.4%** test accuracy with this simple CNN! With a better CNN architecture, we could improve that even more - in this official Keras MNIST CNN example, they achieve **99%** test accuracy after 15 epochs. That's a *really* good accuracy.

> *Unfamiliar with Keras? Read my tutorials on building your first Neural Network with Keras or implementing CNNs with Keras.*

All code from this post is available on Github.

What Now?

We're done! In this 2-part series, we did a full walkthrough of Convolutional Neural Networks, including what they are, how they work, why they're useful, and how to train them. This is just the beginning, though. There's a lot more you could do:

- Read the rest of my Neural Networks from Scratch series.
- Experiment with bigger / better CNNs using proper ML libraries like Tensorflow, Keras, or PyTorch.
- Learn about using Batch Normalization with CNNs.
- Understand how **Data Augmentation** can be used to improve image training sets.
- Read about the ImageNet project and its famous Computer Vision contest, the ImageNet Large Scale Visual Recognition Challenge (ILSVRC).

This blog is open-source on Github.

For Beginners

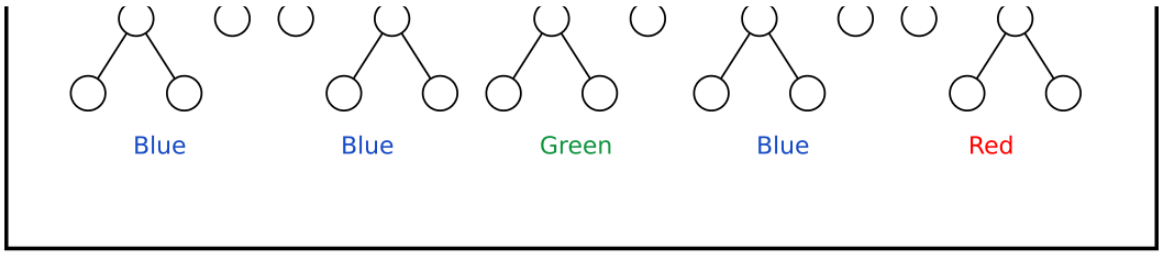
T
o
C

YOU MIGHT ALSO LIKE

Keras for Beginners: Implementing a Convolutional Neural Network

November 10, 2020

A beginner-friendly guide on using Keras to implement a simple Convolutional Neural Network (CNN) in Python.



Random Forests for Complete Beginners

January 13, 2026

The definitive guide to Random Forests and Decision Trees.

Victor Zhou @victorczhou

Software Engineer. I blog about web development, machine learning, and more topics.

SHARE THIS POST

Facebook Twitter LinkedIn Reddit

DISCUSS ON

Twitter Hacker News